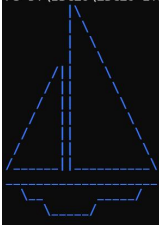


1. Installing Istio and Deploying the Bookinfo Application

1.1. Install Istio

```
PS D:\Istio\istio-1.30.0-beta.0-win-amd64\istio-1.30.0-beta.0> istioctl install -f samples/bookinfo/demo-profile-no-gateways.yaml -y
```



```
✓ Istio core installed 🚀
✓ Istiod installed 🌸
- Pruning removed resources
o-system.
Removed apps/v1, Kind=Deployment/istio-egressgateway.istio-system.
Removed /v1, Kind=Service/istio-ingressgateway.istio-system.
Removed /v1, Kind=Service/istio-egressgateway.istio-system.
Removed /v1, Kind=ServiceAccount/istio-ingressgateway-service-account.istio-system.
Removed /v1, Kind=ServiceAccount/istio-egressgateway-service-account.istio-system.
Removed rbac.authorization.k8s.io/v1, Kind=RoleBinding/istio-ingressgateway-sds.istio-system.
Removed rbac.authorization.k8s.io/v1, Kind=RoleBinding/istio-egressgateway-sds.istio-system.
Removed rbac.authorization.k8s.io/v1, Kind=Role/istio-ingressgateway-sds.istio-system.
Removed rbac.authorization.k8s.io/v1, Kind=Role/istio-egressgateway-sds.istio-system.
✓ Installation complete
PS D:\Istio\istio-1.30.0-beta.0-win-amd64\istio-1.30.0-beta.0>
```

Istio was installed using the demo profile without the default gateway services. The `istioctl` install command installed the Istio core components and Istiod. The installation completed successfully, which prepared the Kubernetes cluster for service mesh features such as traffic management, sidecar injection, and observability.

1.2. Enable Istio Sidecar Injection

```
PS D:\Istio\istio-1.30.0-beta.0-win-amd64\istio-1.30.0-beta.0> kubectl label namespace default istio-injection=enabled
namespace/default labeled
```

```
PS D:\Istio\istio-1.30.0-beta.0-win-amd64\istio-1.30.0-beta.0> kubectl get namespace default --show-labels
```

NAME	STATUS	AGE	LABELS
default	Active	20d	istio-injection=enabled,kubernetes.io/metadata.name=default

The default namespace was labeled with `istio-injection=enabled` to enable automatic Istio sidecar injection. With this label, Istio automatically injects an Envoy sidecar proxy into application Pods created in the default namespace. This allows the applications to participate in the Istio service mesh.

1.3. Install Kubernetes Gateway API CRDs

```
PS D:\Istio\istio-1.30.0-beta.0-win-amd64\istio-1.30.0-beta.0> kubectl kustomize "github.com/kubernetes-sigs/gateway-api
/config/crd?ref=v1.4.0" | kubectl apply -f -
customresourcedefinition.apiextensions.k8s.io/backendtlspolicies.gateway.networking.k8s.io created
customresourcedefinition.apiextensions.k8s.io/gatewayclasses.gateway.networking.k8s.io created
customresourcedefinition.apiextensions.k8s.io/gateways.gateway.networking.k8s.io created
customresourcedefinition.apiextensions.k8s.io/grpcroutes.gateway.networking.k8s.io created
customresourcedefinition.apiextensions.k8s.io/httproutes.gateway.networking.k8s.io created
customresourcedefinition.apiextensions.k8s.io/referencegrants.gateway.networking.k8s.io created
PS D:\Istio\istio-1.30.0-beta.0-win-amd64\istio-1.30.0-beta.0> kubectl get crd gateways.gateway.networking.k8s.io
NAME                                CREATED AT
gateways.gateway.networking.k8s.io  2026-05-03T14:29:52Z
PS D:\Istio\istio-1.30.0-beta.0-win-amd64\istio-1.30.0-beta.0> kubectl get crd httproutes.gateway.networking.k8s.io
NAME                                CREATED AT
httproutes.gateway.networking.k8s.io 2026-05-03T14:29:53Z
PS D:\Istio\istio-1.30.0-beta.0-win-amd64\istio-1.30.0-beta.0>
```

The Kubernetes Gateway API CRDs were installed before creating the Bookinfo Gateway resources. These CRDs allow Kubernetes to recognize Gateway and HTTPRoute resources. After applying the CRD manifests, the gateway and HTTPRoute CRDs were checked to confirm that the required custom resource definitions were available in the cluster.

1.4. Deploy the Bookinfo Sample Application

```
PS D:\Istio\istio-1.30.0-beta.0-win-amd64\istio-1.30.0-beta.0> kubectl apply -f samples/bookinfo/platform/kube/bookinfo.
yaml
service/details created
serviceaccount/bookinfo-details created
deployment.apps/details-v1 created
service/ratings created
serviceaccount/bookinfo-ratings created
deployment.apps/ratings-v1 created
service/reviews created
serviceaccount/bookinfo-reviews created
deployment.apps/reviews-v1 created
deployment.apps/reviews-v2 created
deployment.apps/reviews-v3 created
service/productpage created
serviceaccount/bookinfo-productpage created
deployment.apps/productpage-v1 created
```

The Bookinfo sample application was deployed to the default namespace using the provided Kubernetes manifest. The deployment created several services and deployments, including details, productpage, ratings, and reviews. After the deployment, the `kubectl get services` and `kubectl get pods` commands were used to verify the application resources.

```
PS D:\Istio\istio-1.30.0-beta.0-win-amd64\istio-1.30.0-beta.0> kubectl get services
NAME            TYPE        CLUSTER-IP    EXTERNAL-IP  PORT(S)    AGE
details         ClusterIP   10.100.155.75 <none>       9080/TCP    6s
kubernetes      ClusterIP   10.96.0.1     <none>       443/TCP    20d
productpage     ClusterIP   10.107.199.106 <none>       9080/TCP    6s
ratings         ClusterIP   10.99.134.83  <none>       9080/TCP    6s
reviews         ClusterIP   10.111.52.132 <none>       9080/TCP    6s
PS D:\Istio\istio-1.30.0-beta.0-win-amd64\istio-1.30.0-beta.0> kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
details-v1-764c46cfdb-hqz4s        1/2     PodInitializing   0          11s
productpage-v1-85664dccbc-v6tfm    1/2     PodInitializing   0          10s
ratings-v1-779fdc7f86-5f2sm        1/2     PodInitializing   0          11s
reviews-v1-85964f9f98-xfsdl        1/2     PodInitializing   0          11s
reviews-v2-6f7fbd6c6fd-gq2j5      1/2     PodInitializing   0          11s
reviews-v3-689b477554-9hz95        1/2     PodInitializing   0          11s
PS D:\Istio\istio-1.30.0-beta.0-win-amd64\istio-1.30.0-beta.0> kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
details-v1-764c46cfdb-hqz4s        2/2     Running    0          2m22s
productpage-v1-85664dccbc-v6tfm    2/2     Running    0          2m21s
ratings-v1-779fdc7f86-5f2sm        2/2     Running    0          2m22s
reviews-v1-85964f9f98-xfsdl        2/2     Running    0          2m22s
reviews-v2-6f7fbd6c6fd-gq2j5      2/2     Running    0          2m22s
reviews-v3-689b477554-9hz95        2/2     Running    0          2m22s
PS D:\Istio\istio-1.30.0-beta.0-win-amd64\istio-1.30.0-beta.0>
```

The Bookinfo sample application was deployed to the default namespace. The `kubectl get services` command shows that the `details`, `productpage`, `ratings`, and `reviews` services were created successfully. After the deployment, the `kubectl get pods` command was used to check the Pod status. At first, the Pods were in the `PodInitializing` state with `READY 1/2`, which means that one of the two containers in each Pod was still being initialized. After waiting for the initialization process to complete, the Pods changed to the `Running` state with `READY 2/2`. This confirms that both the application containers and the Istio Envoy sidecar containers were running successfully.

1.5. Verify the Bookinfo Application inside the Cluster

```
PS D:\Istio\istio-1.30.0-beta.0-win-amd64\istio-1.30.0-beta.0> $RATINGS_POD = kubectl get pod -l app=ratings -o jsonpath="{.items[0].metadata.name}"
PS D:\Istio\istio-1.30.0-beta.0-win-amd64\istio-1.30.0-beta.0> echo $RATINGS_POD
ratings-v1-779fcd7f86-5f2sm
PS D:\Istio\istio-1.30.0-beta.0-win-amd64\istio-1.30.0-beta.0> kubectl exec $RATINGS_POD -c ratings -- curl -sS productpage:9080/productpage

<meta charset="utf-8">
<meta http-equiv="X-UA-Compatible" content="IE=edge">
<meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>Simple Bookstore App</title>
<script src="static/tailwind/tailwind.css"></script>
<script type="text/javascript">
  window.addEventListener("DOMContentLoaded", (event) => {
    const dialog = document.querySelector("#dialog");
    const showButton = document.querySelector("#sign-in-button");
    const closeButton = document.querySelector("#close-dialog");

    if (showButton) {
      showButton.addEventListener("click", () => {
        dialog.showModal();
      });
    }

    if (closeButton) {
      closeButton.addEventListener("click", () => {
        dialog.close();
      });
    }
  })
</script>
```

The Bookinfo application was verified from inside the Kubernetes cluster. The name of the `ratings` Pod was first stored in a PowerShell variable. Then, a `curl` request was executed inside the `ratings` container to access the `productpage` service. The returned HTML title showed `Simple Bookstore App`, confirming that the Bookinfo application was running correctly inside the cluster.

```
PS D:\Istio\istio-1.30.0-beta.0-win-amd64\istio-1.30.0-beta.0> kubectl exec $RATINGS_POD -c ratings -- curl -sS productpage:9080/productpage | Select-String "<title>"

<title>Simple Bookstore App</title>
```

1.6. Create and Configure the Bookinfo Gateway

```
PS D:\Istio\istio-1.30.0-beta.0-win-amd64\istio-1.30.0-beta.0> kubectl apply -f samples/bookinfo/gateway-api/bookinfo-ga
teway.yaml
gateway.gateway.networking.k8s.io/bookinfo-gateway created
httproute.gateway.networking.k8s.io/bookinfo created
PS D:\Istio\istio-1.30.0-beta.0-win-amd64\istio-1.30.0-beta.0> kubectl annotate gateway bookinfo-gateway networking.isti
o.io/service-type=ClusterIP --namespace=default
gateway.gateway.networking.k8s.io/bookinfo-gateway annotated
PS D:\Istio\istio-1.30.0-beta.0-win-amd64\istio-1.30.0-beta.0> kubectl get gateway
NAME          CLASS          ADDRESS                                     PROGRAMMED  AGE
bookinfo-gateway  istio  bookinfo-gateway-istio.default.svc.cluster.local  True        27s
```

A Kubernetes Gateway and HTTPRoute were created for the Bookinfo application using the provided gateway manifest. The gateway was annotated to use the ClusterIP service type because the application would be accessed through port forwarding. The `kubectl get gateway` command confirmed that the Bookinfo Gateway was created and programmed successfully.

1.7. Access the Bookinfo Application through Port Forwarding

```
bookinfo-gateway 27s bookinfo-gateway 27s
PS D:\Istio\istio-1.30.0-beta.0-win-amd64\istio-1.30.0-beta.0> kubectl port-forward svc/bookinfo-gateway-istio 8080:80
Forwarding from 127.0.0.1:8080 -> 80
Forwarding from [::1]:8080 -> 80
Handling connection for 8080
Handling connection for 8080
```

Port forwarding was used to access the Bookinfo Gateway from the local machine. The `kubectl port-forward` command forwarded local port 8080 to port 80 of the `bookinfo-gateway-istio` Service. After the port forwarding process started, the Bookinfo product page was opened in the browser through `http://localhost:8080/productpage`.

BookInfo Sample

Sign In

The Comedy of Errors

Wikipedia Summary: The Comedy of Errors is one of **William Shakespeare's** early plays. It is his shortest and one of his most farcical comedies, with a major part of the humour coming from slapstick and mistaken identity, in addition to puns and word play.

[Learn more about Istio](#)

Book Details

ISBN-10	Publisher	Pages	Type	Language
1234567890	PublisherA	200	paperback	English

Book Reviews

★★★★★

"An extremely entertaining play by Shakespeare. The slapstick humour is refreshing!"

Reviewer1
Reviews served by: reviews-v2-6f7fbd6cf6-ga2j5

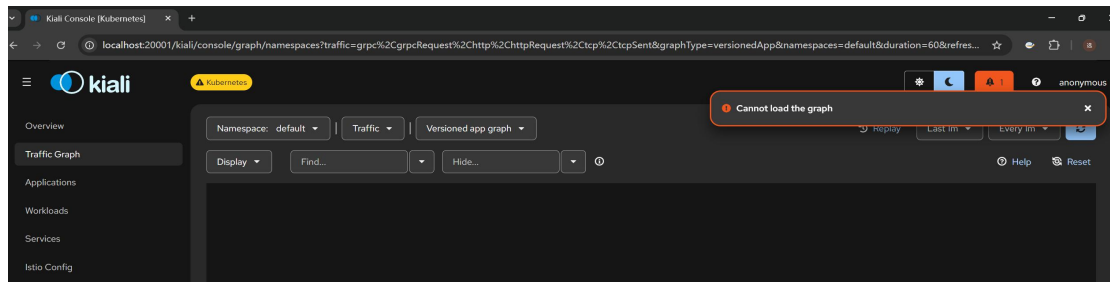
★★★★☆

"Absolutely fun and entertaining. The play lacks thematic depth when compared to other plays by Shakespeare."

Reviewer2
Reviews served by: reviews-v2-6f7fbd6cf6-ga2j5

The Bookinfo product page was displayed successfully in the browser. This confirmed that the Gateway and HTTPRoute were configured correctly and that external traffic could reach the Bookinfo application through the local port-forwarding connection.

1.8. View the Kiali Dashboard



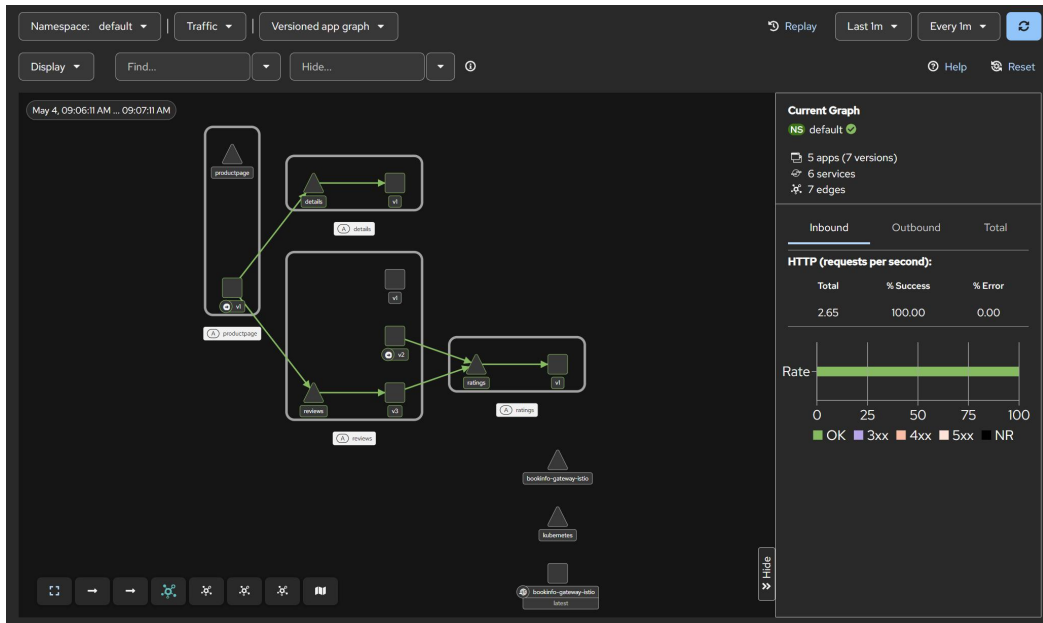
After opening the Kiali dashboard, the Traffic Graph page failed to load and displayed an error message indicating that Kiali could not query Prometheus. This showed that the Kiali web interface was accessible, but the metrics backend required for traffic visualization was not working correctly.

```
PS C:\Users\27900> kubectl get pods -n istio-system
NAME                                READY   STATUS    RESTARTS   AGE
istiod-6d6c59776f-2tghx            1/1     Running   0           10h
kiali-8676f87b85-k22mj             1/1     Running   0           10h
```

To investigate the problem, the Pods in the istio-system namespace were checked using the `kubectl get pods -n istio-system` command. The output showed that istiod and kiali were running, but no Prometheus Pod was present. Since Kiali depends on Prometheus to collect and query traffic metrics, the missing Prometheus component explained why the traffic graph could not be loaded.

```
PS D:\Istio\istio-1.30.0-beta.0-win-amd64\istio-1.30.0-beta.0> kubectl apply -f samples/addons/prometheus.yaml
serviceaccount/prometheus created
configmap/prometheus created
clusterrole.rbac.authorization.k8s.io/prometheus created
clusterrolebinding.rbac.authorization.k8s.io/prometheus created
service/prometheus created
deployment.apps/prometheus created
PS D:\Istio\istio-1.30.0-beta.0-win-amd64\istio-1.30.0-beta.0> kubectl rollout status deployment/prometheus -n istio-system
Waiting for deployment "prometheus" rollout to finish: 0 of 1 updated replicas are available...
deployment "prometheus" successfully rolled out
PS D:\Istio\istio-1.30.0-beta.0-win-amd64\istio-1.30.0-beta.0> kubectl get pods -n istio-system
NAME                                READY   STATUS    RESTARTS   AGE
istiod-6d6c59776f-2tghx            1/1     Running   0           10h
kiali-8676f87b85-k22mj             1/1     Running   0           10h
prometheus-65c465688f-6h544       2/2     Running   0           27s
PS D:\Istio\istio-1.30.0-beta.0-win-amd64\istio-1.30.0-beta.0>
```

The working directory was changed to the Istio installation directory, where the `samples/addons` folder was available. Prometheus was then installed using the `kubectl apply -f samples/addons/prometheus.yaml` command. After the installation, the Prometheus deployment status was checked to confirm that the metrics component was running in the istio-system namespace.



After Prometheus was installed successfully, traffic was generated by sending multiple requests to the Bookinfo product page. The Kiali dashboard was refreshed and the Traffic Graph page displayed the Bookinfo service mesh in the default namespace. The graph showed the relationships among productpage, details, reviews, and ratings services. The traffic statistics showed a 100% success rate, confirming that Kiali could successfully access the metrics backend and visualize the traffic flow between the Bookinfo services.

1.9. Clean Up Istio and Bookinfo Resources

```
PS D:\Istio\istio-1.30.0-beta.0-win-amd64\istio-1.30.0-beta.0> kubectl delete -f samples/bookinfo/platform/kube/bookinfo
.yaml
service "details" deleted from default namespace
serviceaccount "bookinfo-details" deleted from default namespace
deployment.apps "details-v1" deleted from default namespace
service "ratings" deleted from default namespace
serviceaccount "bookinfo-ratings" deleted from default namespace
deployment.apps "ratings-v1" deleted from default namespace
service "reviews" deleted from default namespace
serviceaccount "bookinfo-reviews" deleted from default namespace
deployment.apps "reviews-v1" deleted from default namespace
deployment.apps "reviews-v2" deleted from default namespace
deployment.apps "reviews-v3" deleted from default namespace
service "productpage" deleted from default namespace
serviceaccount "bookinfo-productpage" deleted from default namespace
deployment.apps "productpage-v1" deleted from default namespace
PS D:\Istio\istio-1.30.0-beta.0-win-amd64\istio-1.30.0-beta.0> kubectl delete -f samples/bookinfo/gateway-api/bookinfo-g
ateway.yaml
gateway.gateway.networking.k8s.io "bookinfo-gateway" deleted from default namespace
httproute.gateway.networking.k8s.io "bookinfo" deleted from default namespace
PS D:\Istio\istio-1.30.0-beta.0-win-amd64\istio-1.30.0-beta.0> kubectl delete -f samples/addons/kiali.yaml
serviceaccount "kiali" deleted from istio-system namespace
configmap "kiali" deleted from istio-system namespace
clusterrole.rbac.authorization.k8s.io "kiali" deleted
clusterrolebinding.rbac.authorization.k8s.io "kiali" deleted
service "kiali" deleted from istio-system namespace
deployment.apps "kiali" deleted from istio-system namespace
```

After completing the Istio and Bookinfo experiment, the Bookinfo application, Gateway resources, and Istio components were removed from the Kubernetes cluster. The cleanup commands deleted the sample application resources and uninstalled Istio from the cluster. Finally,

the remaining resources were checked to confirm that the experimental environment had been cleaned up.